```
58        else cout << ret << endl;
59
60        return 0;
61    }
```

3.3 【模板】最小生成树

> **题目来源：** 洛谷
>
> **题目链接：** P3366 【模板】最小生成树
>
> **难度系数：** ★★

【题目描述】

> 如题，给出一个无向图，求出最小生成树，如果该图不连通，则输出 `orz`。

【输入描述】

> 第一行包含两个整数 N, M，表示该图共有 N 个结点和 M 条无向边。
>
> 接下来 M 行每行包含三个整数 X_i, Y_i, Z_i，表示有一条长度为 Z_i 的无向边连接结点 X_i, Y_i。

【输出描述】

> 如果该图连通，则输出一个整数表示最小生成树的各边的长度之和。如果该图不连通则输出 `orz`。

【示例一】

> 输入：
>
> ```
> 4 5
> 1 2 2
> 1 3 2
> 1 4 3
> 2 3 4
> 3 4 3
> ```
>
> 输出：
>
> ```
> 7
> ```

解法：

参考前两个板块里面的代码。

3.4 买礼物

题目来源： 洛谷

题目链接： P1194 买礼物

难度系数： ★★

【题目描述】

又到了一年一度的明明生日了，明明想要买 B 样东西，巧的是，这 B 样东西价格都是 A 元。

但是，商店老板说最近有促销活动，也就是：

如果你买了第 I 样东西，再买第 J 样，那么就可以只花 $K_{I,J}$ 元，更巧的是，$K_{I,J}$ 竟然等于 $K_{J,I}$ 。

现在明明想知道，他最少要花多少钱。

【输入描述】

第一行两个整数，A, B。

接下来 B 行，每行 B 个数，第 I 行第 J 个为 $K_{I,J}$。

我们保证 $K_{I,J} = K_{J,I}$ 并且 $K_{I,I} = 0$。

特别的，如果 $K_{I,J} = 0$，那么表示这两样东西之间不会导致优惠。

注意 $K_{I,J}$ **可能大于** A。

【输出描述】

一个整数，为最小要花的钱数。

【示例一】

输入：

```
1 1
0
```

输出：

```
1
```

【示例二】

输入：

```
3 3
0 2 4
2 0 2
4 2 0
```

输出：

```
7
```

【解法】

题目转化：

- 如果把每一个零食看成一个节点，优惠看成一条边，就变成在图中找最小生成树的问题。
- 因此，跑一遍 kk 算法即可。

注意：

1. 存边的时候，没有必要存重复的，并且权值过大的也不需要存；
2. 最终提取结果的时候，虽然有可能构造不出来一棵最小生成树，但是要在已有的构造情况下处理结果。

【参考代码】

```
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 500 * 500 + 10;

int a, n;

int pos;
struct node
{
    int x, y, z;
}e[N];

int fa[N]; // 并查集

int find(int x)
{
    return fa[x] == x ? x : fa[x] = find(fa[x]);
}

int cnt, ret;

bool cmp(node& a, node& b)
{
    return a.z < b.z;
}

```

```
30  void kk()
31  {
32      sort(e + 1, e + 1 + pos, cmp);
33
34      for(int i = 1; i <= pos; i++)
35      {
36          int x = e[i].x, y = e[i].y, z = e[i].z;
37
38          int fx = find(x), fy = find(y);
39          if(fx != fy)
40          {
41              cnt++;
42              ret += z;
43              fa[fx] = fy;
44          }
45      }
46  }
47
48  int main()
49  {
50      cin >> a >> n;
51      for(int i = 1; i <= n; i++)
52      {
53          for(int j = 1; j <= n; j++)
54          {
55              int k; cin >> k;
56              if(i >= j || k > a || k == 0) continue;
57              pos++;
58              e[pos].x = i; e[pos].y = j; e[pos].z = k;
59          }
60      }
61
62      for(int i = 1; i <= n; i++) fa[i] = i;
63
64      kk();
65
66      cout << ret + (n - cnt) * a << endl;
67
68      return 0;
69  }
```

3.5 繁忙的都市

> **题目来源： 洛谷**

> 题目链接： P2330 [SCOI2005] 繁忙的都市
>
> 难度系数： ★★

【题目描述】

> 城市 C 是一个非常繁忙的大都市，城市中的道路十分的拥挤，于是市长决定对其中的道路进行改造。城市 C 的道路是这样分布的：城市中有 n 个交叉路口，有些交叉路口之间有道路相连，两个交叉路口之间最多有一条道路相连接。这些道路是双向的，且把所有的交叉路口直接或间接的连接起来了。每条道路都有一个分值，分值越小表示这个道路越繁忙，越需要进行改造。但是市政府的资金有限，市长希望进行改造的道路越少越好，于是他提出下面的要求：
>
> 1. 改造的那些道路能够把所有的交叉路口直接或间接的连通起来。
> 2. 在满足要求 1 的情况下，改造的道路尽量少。
> 3. 在满足要求 1、2 的情况下，改造的那些道路中分值最大的道路分值尽量小。
>
> 任务：作为市规划局的你，应当作出最佳的决策，选择哪些道路应当被修建。

【输入描述】

> 第一行有两个整数 n, m 表示城市有 n 个交叉路口， m 条道路。
>
> 接下来 m 行是对每条道路的描述， u, v, c 表示交叉路口 u 和 v 之间有道路相连，分值为 c 。

【输出描述】

> 两个整数 s, max ，表示你选出了几条道路，分值最大的那条道路的分值是多少。

【示例一】

> 输入：
>
> ```
> 4 5
> 1 2 3
> 1 4 5
> 2 4 7
> 2 3 6
> 3 4 8
> ```
>
> 输出：
>
> ```
> 3 6
> ```

【解法】

定理：最小生成树就是瓶颈生成树。

在 kk 算法中，维护边权的最大值即可。

【参考代码】

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 310, M = 8010;

int n, m;
struct node
{
    int x, y, z;
}e[M];

int fa[N]; // 并查集

int find(int x)
{
    return fa[x] == x ? x : fa[x] = find(fa[x]);
}

int ret; // 最大边的权值

bool cmp(node& a, node& b)
{
    return a.z < b.z;
}

void kk()
{
    // 初始化并查集
    for(int i = 1; i <= n; i++) fa[i] = i;

    sort(e + 1, e + 1 + m, cmp);

    for(int i = 1; i <= m; i++)
    {
        int x = e[i].x, y = e[i].y, z = e[i].z;

        int fx = find(x), fy = find(y);
        if(fx != fy)
        {
            ret = max(ret, z);
            fa[fx] = fy;
        }
    }
}
```

```
47
48  int main()
49  {
50      cin >> n >> m;
51      for(int i = 1; i <= m; i++) cin >> e[i].x >> e[i].y >> e[i].z;
52
53      cout << n - 1 << " ";
54
55      kk();
56
57      cout << ret << endl;
58
59      return 0;
60  }
```

3.6 滑雪

题目来源： 洛谷

题目链接： P2573 [SCOI2012] 滑雪

难度系数： ★★★

【题目描述】

a180285 非常喜欢滑雪。他来到一座雪山，这里分布着 m 条供滑行的轨道和 n 个轨道之间的交点（同时也是景点），而且每个景点都有一编号 $i(1 \leq i \leq n)$ 和一高度 h_i。

a180285 能从景点 i 滑到景点 j 当且仅当存在一条 i 和 j 之间的边，且 i 的高度**不小于** j。与其他滑雪爱好者不同，a180285 喜欢用最短的滑行路径去访问尽量多的景点。如果仅仅访问一条路径上的景点，他会觉得数量太少。

于是 a180285 拿出了他随身携带的时间胶囊。这是一种很神奇的药物，吃下之后可以立即回到上个经过的景点（不用移动也不被认为是 a180285 滑行的距离）。

请注意，这种神奇的药物是可以连续食用的，即能够回到较长时间之前到过的景点（比如上上个经过的景点和上上上个经过的景点）。 现在，a180285站在 1 号景点望着山下的目标，心潮澎湃。他十分想知道在不考虑时间胶囊消耗的情况下，以最短滑行距离滑到尽量多的景点的方案（即满足经过景点数最大的前提下使得滑行总距离最小）。你能帮他求出最短距离和景点数吗?

【输入描述】

输入的第一行是两个整数 n, m。 接下来一行有 n 个整数 h_i，分别表示每个景点的高度。

接下来 m 行，表示各个景点之间轨道分布的情况。每行三个整数 u, v, k，表示编号为 u 的景点和编号为 v 的景点之间有一条长度为 k 的轨道。

【输出描述】

> 输出一行，表示 a180285 最多能到达多少个景点，以及此时最短的滑行距离总和。

【示例一】

> 输入：
>
> 3 3
>
> 3 2 1
>
> 1 2 1
>
> 2 3 1
>
> 1 3 10
>
> 输出：
>
> 3 2

【解法】

第一问：从起点开始，做一次 dfs/bfs 就可以扫描到所有的点。

第二问：因为有回溯的效果，相当于就是选择一些边，把所有的点都连接起来。但是需要注意：

- 由于这些边是有方向的，我们只要保证能从 1 位置出发，访问到所有的点即可。与最小生成树还是有差异的。
- 为了能保证选出来的边能够从 1 访问所有点，应该优先考虑去往更高位置的边，这样才能向下走到更低的位置。

【参考代码】

```
#include <iostream>
#include <algorithm>
#include <cstring>
#include <vector>

using namespace std;

typedef long long LL;
typedef pair<int, int> PII;
const int N = 1e5 + 10, M = 2e6 + 10;

int n, m;
int h[N];

```

```cpp
vector<PII> edges[N]; // 存图

int fa[N]; // 并查集

int find(int x)
{
    return fa[x] == x ? x : fa[x] = find(fa[x]);
}

LL cnt, ret;
bool st[N]; // 标记 dfs 的过程中，哪些点已经遍历过

int pos;
struct node
{
    int x, y, z;
}e[M];

void dfs(int u)
{
    cnt++;
    st[u] = true;

    for(auto& p : edges[u])
    {
        int v = p.first, k = p.second;
        pos++;
        e[pos].x = u; e[pos].y = v; e[pos].z = k;

        if(!st[v]) dfs(v);
    }
}

bool cmp(node& a, node& b)
{
    // 一定要先考虑高度，再考虑长度
    int y1 = a.y, z1 = a.z, y2 = b.y, z2 = b.z;

    if(h[y1] != h[y2]) return h[y1] > h[y2];
    else return z1 < z2;
}

void kk()
{
    for(int i = 1; i <= n; i++) fa[i] = i;

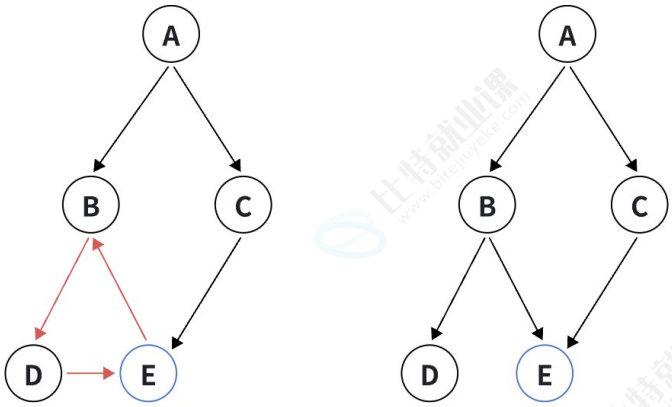
    sort(e + 1, e + 1 + pos, cmp);
```

```
62
63      for(int i = 1; i <= pos; i++)
64      {
65          int x = e[i].x, y = e[i].y, z = e[i].z;
66
67          int fx = find(x), fy = find(y);
68          if(fx != fy)
69          {
70              ret += z;
71              fa[fx] = fy;
72          }
73      }
74  }
75
76  int main()
77  {
78      cin >> n >> m;
79      for(int i = 1; i <= n; i++) cin >> h[i];
80      for(int i = 1; i <= m; i++)
81      {
82          int x, y, z; cin >> x >> y >> z;
83          if(h[x] >= h[y]) edges[x].push_back({y, z});
84          if(h[x] <= h[y]) edges[y].push_back({x, z});
85      }
86
87      dfs(1);
88      cout << cnt << " ";
89
90      kk();
91      cout << ret << endl;
92
93      return 0;
94  }
```

4. 拓扑排序

4.1 有向无环图

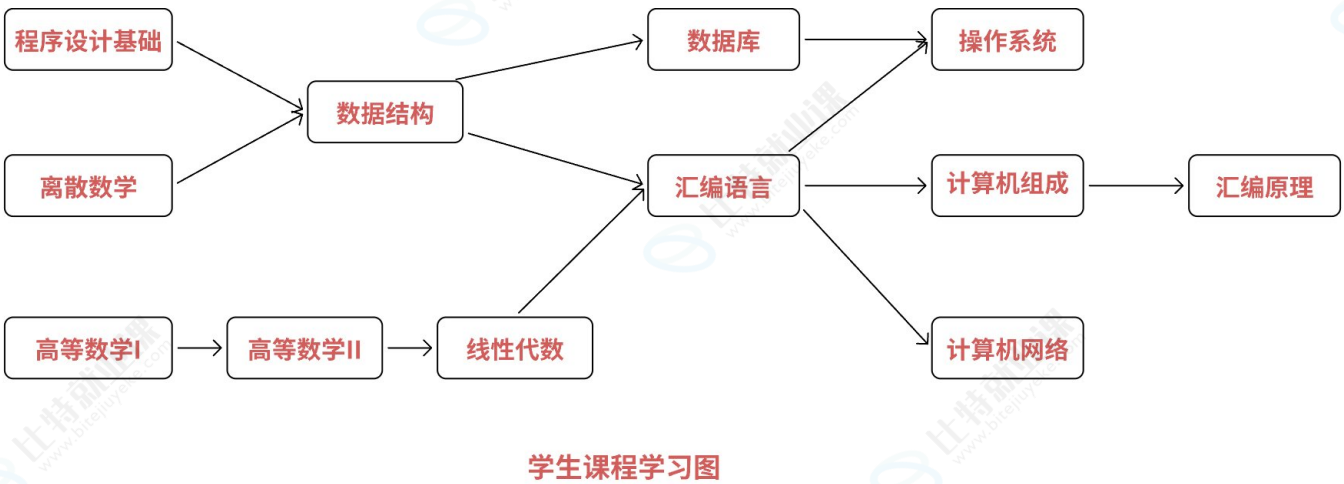
若一个有向图中不存在回路，则称为有向无环图(directed acycline graph)，简称 DAG 图。



有向图　　有向无环图(DAG图)

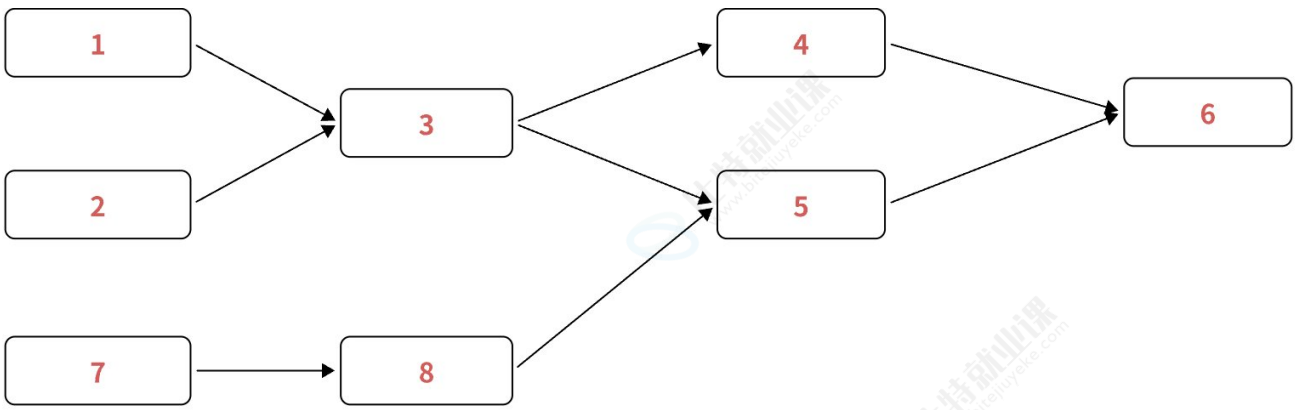
4.2 AOV 网

举一个现实中的例子：课程的学习是有优先次序的，如果规划不当会严重影响学习效果。课程间的先后次序可以用有向图表示：



学生课程学习图

在这种有向图中，用顶点表示活动，用有向边 $<V_i, V_j>$ 表示活动 V_i 必须先于活动 V_j 进行，这种有向图叫做顶点表示活动的网络(Activity On Vertex Network)，简称 AOV 网。



AOV 网中不能有回路，否则就不能确定回路中的活动究竟哪个先实施。因此一个可行的 AOV 网必须是有向无环图。

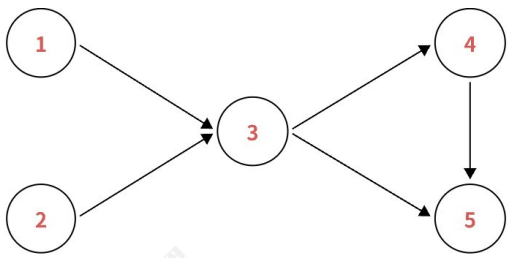
4.3 拓扑排序

拓扑排序的目标是将有向无环图中的所有结点排序，使得排在前面的结点不能依赖于排在后面的结点。在课程问题中，相当于就是找到一个排课的合法顺序。具体流程可借助队列进行：

1. 将图中所有入度为 0 的点，加入到队列中；
2. 取出队头元素，删除与该点相连的边。如果删除之后的后继结点的入度变为 0，加入到队列中；
3. 重复 2 操作，直到图中没有点或者没有入度为 0 的点为止。

拓扑排序判断是否有环：

跑一遍拓扑排序算法，如果有结点没有进队，那么就表明有环。



4.4 【模板】拓扑排序

> 题目来源： 洛谷
>
> 题目链接： B3644 【模板】拓扑排序 / 家谱树

> **难度系数：** ★★

【题目描述】

> 有个人的家族很大，辈分关系很混乱，请你帮整理一下这种关系。给出每个人的后代的信息。输出一个序列，使得每个人的后辈都比那个人后列出。

【输入描述】

> 第 1 行一个整数 N（$1 \leq N \leq 100$），表示家族的人数。接下来 N 行，第 i 行描述第 i 个人的后代编号 $a_{i,j}$，表示 $a_{i,j}$ 是 i 的后代。每行最后是 0 表示描述完毕。

【输出描述】

> 输出一个序列，使得每个人的后辈都比那个人后列出。如果有多种不同的序列，输出任意一种即可。

【示例一】

> 输入：
>
> ```
> 5
> 0
> 4 5 1 0
> 1 0
> 5 3 0
> 3 0
> ```
>
> 输出：
>
> ```
> 2 4 5 3 1
> ```

【解法】

模板题~

【参考代码】

```cpp
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

const int N = 110;

int n;
vector<int> edges[N];
int in[N]; // 存入度信息
```

```cpp

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        int j;
        while(cin >> j, j)
        {
            edges[i].push_back(j);
            in[j]++; // 统计入度信息
        }
    }

    // 拓扑排序
    queue<int> q;
    for(int i = 1; i <= n; i++)
    {
        if(in[i] == 0) q.push(i);
    }

    while(q.size())
    {
        int x = q.front(); q.pop();
        cout << x << " ";

        // 删除对应的边
        for(auto y : edges[x])
        {
            in[y]--;
            if(in[y] == 0) q.push(y);
        }
    }


    return 0;
}
```

4.5 摄像头

> **题目来源：** 洛谷
>
> **题目链接：** P2712 摄像头
>
> **难度系数：** ★★

【题目描述】

> 食品店里有 n 个摄像头，这种摄像头很笨拙，只能拍摄到固定位置。现有一群胆大妄为的松鼠想要抢劫食品店，为了不让摄像头拍下他们犯罪的证据，他们抢劫前的第一件事就是砸毁这些摄像头。
>
> 为了便于砸毁摄像头，松鼠歹徒们把所有摄像头和摄像头能监视到的地方统一编号，一个摄像头能被砸毁的条件是该摄像头所在位置不被其他摄像头监视。
>
> 现在你的任务是帮松鼠们计算是否可以砸掉所有摄像头，如不能则输出还没砸掉的摄像头的数量。

【输入描述】

> 第 1 行，一个整数 n ，表示摄像头的个数。
>
> 第 2 到 $n+1$ 行是摄像头的信息，包括：摄像头的位置 x ，以及这个摄像头可以监视到的位置数 m ，之后 m 个数 y 是此摄像头可以监视到的位置。(砸了这些摄像头之后自然这些位置就监视不到了)

【输出描述】

> 若可以砸掉所有摄像头则输出 `"YES"` ，否则输出还没砸掉的摄像头的数量。（不带引号）

【示例一】

> 输入：
>
> ```
> 5
> 1 1 2
> 2 1 1
> 3 1 7
> 4 1 1
> 5 0
> ```
>
> 输出：
>
> ```
> 2
> ```

【解法】

拓扑排序判断是否有环。

直接跑一遍拓扑排序，然后统计一下有多少摄像头没有出队。那么这些没有出队的摄像头就是环里面的元素。

注意：

- 有些位置可能没有摄像头，需要判断一下。

【参考代码】

```cpp
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

const int N = 510;

int n;
vector<int> edges[N];
int in[N];
bool st[N]; // 标记哪个位置有摄像头

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        int x, m, y; cin >> x >> m;
        st[x] = true;
        while(m--)
        {
            cin >> y;
            edges[x].push_back(y);
            in[y]++;
        }
    }

    queue<int> q;
    // 1. 入度为 0 的点加入到队列中
    for(int i = 0; i <= 500; i++)
    {
        if(st[i] && in[i] == 0) q.push(i);
    }

    while(q.size())
    {
        auto x = q.front(); q.pop();

        for(auto y : edges[x])
        {
            in[y]--;
            if(st[y] && in[y] == 0) q.push(y);
        }
    }

```

```
47        int ret = 0;
48        for(int i = 0; i <= 500; i++)
49        {
50            if(st[i] && in[i]) ret++;
51        }
52
53        if(ret == 0) cout << "YES" << endl;
54        else cout << ret << endl;
55
56        return 0;
57    }
```

4.6 最大食物链计数

> **题目来源：** 洛谷
>
> **题目链接：** P4017 最大食物链计数
>
> **难度系数：** ★★★

【题目描述】

> 给你一个食物网，你要求出这个食物网中最大食物链的数量。
>
> （这里的“最大食物链”，指的是**生物学意义上的食物链**，即**最左端是不会捕食其他生物的生产者，最右端是不会被其他生物捕食的消费者**。）
>
> Delia 非常急，所以你只有 1 秒的时间。
>
> 由于这个结果可能过大，你只需要输出总数模上 80112002 的结果。

【输入描述】

> 第一行，两个正整数 n、m，表示生物种类 n 和吃与被吃的关系数 m。
>
> 接下来 m 行，每行两个正整数，表示被吃的生物 A 和吃 A 的生物 B。

【输出描述】

> 一行一个整数，为最大食物链数量模上 80112002 的结果。

【示例一】

> 输入：
>
> 5 7
>
> 1 2
>
> 1 3
>
> 2 3

> 3 5
> 2 5
> 4 5
> 3 4
> 输出：
> 5

【解法】

注意审题！题目问的是一共有多少条路径！

拓扑排序的过程中，进行动态规划。

对于每一个节点 i ，通过它的路径为：前驱所有结点的路径总数之和。因此，可以在拓扑排序的过程中，维护从起点开始到达每一个节点的路径总数。

【参考代码】

```cpp
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

const int N = 5010, MOD = 80112002;

int n, m;
vector<int> edges[N];
int in[N], out[N];
int f[N];

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= m; i++)
    {
        int x, y; cin >> x >> y;
        edges[x].push_back(y);

        in[y]++; out[x]++;
    }

    queue<int> q;
    for(int i = 1; i <= n; i++)
```

```
    {
        if(in[i] == 0)
        {
            f[i] = 1;
            q.push(i);
        }
    }

    while(q.size())
    {
        auto x = q.front(); q.pop();

        for(auto y : edges[x])
        {
            // x -> y
            // f[x]
            f[y] = (f[y] + f[x]) % MOD;
            in[y]--;
            if(in[y] == 0) q.push(y);
        }
    }

    int ret = 0;
    for(int i = 1; i <= n; i++)
    {
        if(out[i] == 0)
        {
            ret = (ret + f[i]) % MOD;
        }
    }

    cout << ret << endl;

    return 0;
}
```

4.7 杂物

> **题目来源：** 洛谷
>
> **题目链接：** P1113 杂务
>
> **难度系数：** ★★★

【题目描述】

John 的农场在给奶牛挤奶前有很多杂务要完成，每一项杂务都需要一定的时间来完成它。比如：他们要将奶牛集合起来，将他们赶进牛棚，为奶牛清洗乳房以及一些其它工作。尽早将所有杂务完成是必要的，因为这样才有更多时间挤出更多的牛奶。

当然，有些杂务必须在另一些杂务完成的情况下才能进行。比如：只有将奶牛赶进牛棚才能开始为它清洗乳房，还有在未给奶牛清洗乳房之前不能挤奶。我们把这些工作称为完成本项工作的准备工作。至少有一项杂务不要求有准备工作，这个可以最早着手完成的工作，标记为杂务 1 。

John 有需要完成的 n 个杂务的清单，并且这份清单是有一定顺序的，杂务 $k(k>1)$ 的准备工作只可能在杂务 1 至 $k-1$ 中。

写一个程序依次读入每个杂务的工作说明。计算出所有杂务都被完成的最短时间。当然互相没有关系的杂务可以同时工作，并且，你可以假定 John 的农场有足够多的工人来同时完成任意多项任务。

【输入描述】

第 1 行：一个整数 $n(3 \le n \le 10,000)$ ，必须完成的杂务的数目；

第 2 至 $n+1$ 行，每行有一些用空格隔开的整数，分别表示：

- 工作序号（保证在输入文件中是从 1 到 n 有序递增的）；
- 完成工作所需要的时间 $len(1 \le len \le 100)$ ；
- 一些必须完成的准备工作，总数不超过 100 个，由一个数字 0 结束。有些杂务没有需要准备的工作只描述一个单独的 0 。

保证整个输入文件中不会出现多余的空格。

【输出描述】

一个整数，表示完成所有杂务所需的最短时间。

【示例一】

输入：

```
7
1 5 0
2 2 1 0
3 3 2 0
4 6 1 0
5 1 2 4 0
6 8 2 4 0
7 4 3 5 6 0
```

输出：

```
23
```

【解法】

拓扑排序的过程中，进行动态规划。

对于每一个事件 i ，完成它的最小时间为：完成前驱所有事件的最小时间中的最大值 + 当前事件的完成时间。因此，可以在拓扑排序的过程中，维护每一个事件完成的最小时间，然后更新当前事件的最小时间。

【参考代码】

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

const int N = 10010;

int n;
vector<int> edges[N];
int in[N], f[N];
int len[N];

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        int b, a;
        cin >> b >> len[b];
        while(cin >> a, a)
        {
            edges[a].push_back(b);
            in[b]++;
        }
    }

    queue<int> q;
    for(int i = 1; i <= n; i++)
    {
        if(in[i] == 0) q.push(i);
    }

    int ret = 0;
    while(q.size())
    {
```

```cpp
            int a = q.front(); q.pop();
            f[a] += len[a];
            ret = max(ret, f[a]);

            for(auto b : edges[a])
            {
                f[b] = max(f[b], f[a]);
                in[b]--;
                if(in[b] == 0) q.push(b);
            }
        }

        cout << ret << endl;

        return 0;
    }
```

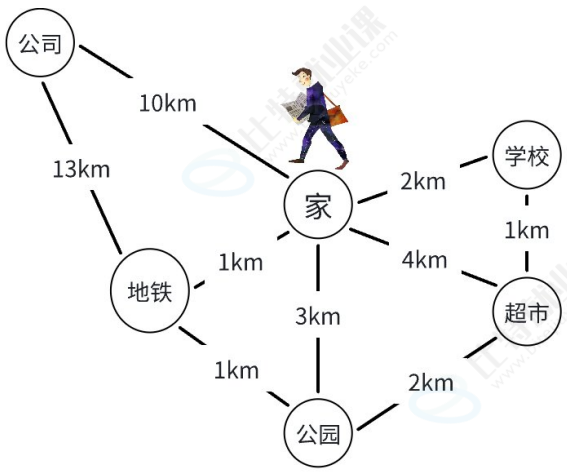
5. 单源最短路

在图 G 中，假设 v_i 和 v_j 为图中的两个顶点，那么 v_i 到 v_j 路径上所经过边的权值之和就称为带权路径长度。

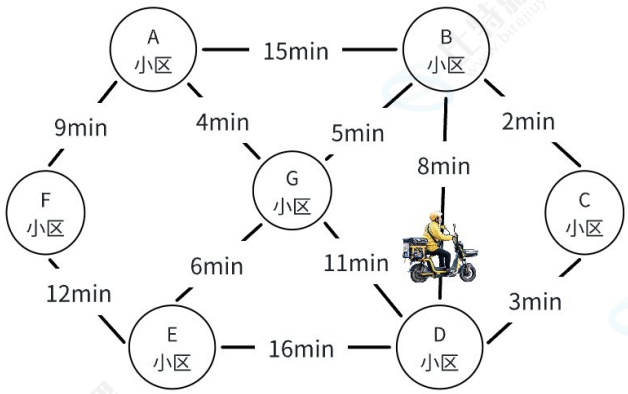
由于 v_i 到 v_j 的路径可能有多条，将带权路径长度最短的那条路径称为最短路径。

最短路一般分为两类：

- **单源最短路**，即图中一个顶点到其它各顶点的最短路径。
- **多源最短路**，即图中每对顶点间的最短路径。



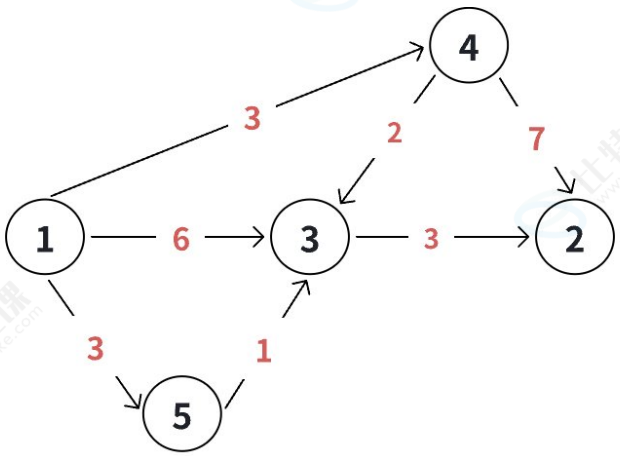
怎么走才能从家到各个地方最近?



任意两个小区间怎么走才最近?

5.1 常规版 dijkstra 算法

Dijkstra 算法是基于贪心思想的单源最短路算法，求解的是"非负权图"上单源最短路径。



常规版 dijkstra 算法流程：

- 准备工作：
  - 创建一个长度为 `n` 的 `dist` 数组，其中 `dist[i]` 表示从起点到 `i` 结点的最短路；
  - 创建一个长度为 `n` 的 `bool` 数组 `st`，其中 `st[i]` 表示 `i` 点是否已经确定了最短路。
- 初始化：`dist[1] = 0`，其余结点的 `dist` 值为无穷大，表示还没有找到最短路。
- 重复：在所有没有确定最短路的点中，找出最短路长度最小的点 `u`。打上确定最短路的标记，然后对 `u` 的出边进行松弛操作；
- 重复上述操作，直到所有点的最短路都确定。

代码实现：（大家会发现，这个代码非常像 prim 算法）

```cpp
#include <iostream>
#include <vector>
#include <cmath>

using namespace std;

typedef pair<int, int> PII;
typedef long long LL;

const int N = 1e4 + 10, INF = 2147483647;
```

```cpp

int n, m, s;
vector<PII> edges[N];

LL dist[N];
bool st[N];

void dijkstra()
{
    for(int i = 0; i <= n; i++) dist[i] = INF;
    dist[s] = 0;

    for(int i = 1; i < n; i++)
    {
        int u = 0;
        for(int j = 1; j <= n; j++)
        {
            if(!st[j] && dist[j] < dist[u]) u = j;
        }

        st[u] = true;

        for(auto& t : edges[u])
        {
            int v = t.first, w = t.second;
            if(dist[u] + w < dist[v])
            {
                dist[v] = dist[u] + w;
            }
        }
    }

    for(int i = 1; i <= n; i++)
    {
        cout << dist[i] << " ";
    }
}

int main()
{
    cin >> n >> m >> s;
    for(int i = 1; i <= m; i++)
    {
        int u, v, w; cin >> u >> v >> w;
        edges[u].push_back({v, w});
    }

```

```
    dijkstra();

    return 0;
}
```

5.2 堆优化版 dijkstra 算法

在常规版的基础上，用优先级队列去维护待确定最短路的结点。

堆优化版的 dijkstra 算法流程：

- 准备工作：
  - 创建一个长度为 `n` 的 `dist` 数组，其中 `dist[i]` 表示从起点到 `i` 结点的最短路；
  - 创建一个长度为 `n` 的 `bool` 数组 `st`，其中 `st[i]` 表示 `i` 点是否已经确定了最短路；
  - 创建一个小根堆，维护更新后的结点。（也就是需要确定最短路的结点）
- 初始化：`dist[1] = 0`，然后将 `{0, s}` 加到堆里；其余结点的 `dist` 值为无穷大，表示还没有找到最短路。
- 重复：弹出堆顶元素，如果该元素已经标记过，就跳过；如果没有标记过，打上标记，进行松弛操作。
- 重复上述操作，直到堆里面没有元素为止。

代码实现：（其实 prim 算法也有一个堆优化的版本，但是时间复杂度和 kk 算法相差无几，因此没有讲解）

```
#include <iostream>
#include <queue>
#include <vector>
#include <cstring>

using namespace std;

typedef pair<int, int> PII;
const int N = 1e5 + 10;

int n, m, s;
```

```cpp
vector<PII> edges[N];

bool st[N];
int dist[N];

void dijkstra()
{
    memset(dist, 0x3f, sizeof dist);
    priority_queue<PII, vector<PII>, greater<PII>> heap; // 小根堆

    heap.push({0, s});
    dist[s] = 0;

    while(heap.size())
    {
        auto t = heap.top(); heap.pop();
        int u = t.second;

        if(st[u]) continue;
        st[u] = true;

        for(auto& t : edges[u])
        {
            int v = t.first, w = t.second;
            if(dist[u] + w < dist[v])
            {
                dist[v] = dist[u] + w;
                heap.push({dist[v], v});
            }
        }
    }

    for(int i = 1; i <= n; i++) cout << dist[i] << " ";
}

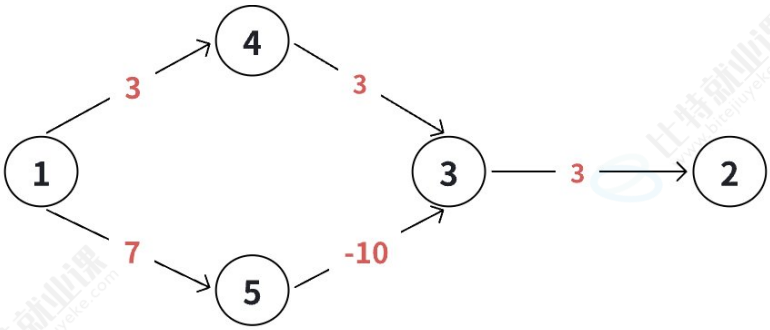
int main()
{
    cin >> n >> m >> s;
    for(int i = 1; i <= m; i++)
    {
        int x, y, z; cin >> x >> y >> z;
        edges[x].push_back({y, z});
    }

    dijkstra();

    return 0;
```

```
59    }
```

如果出现负权边，就会出错：



5.3 bellman-ford 算法

Bellman–Ford 算法(之后简称 BF 算法)是一种基于松弛操作的最短路算法，可以求出有负权的图的最短路，并可以对最短路不存在的情况进行判断。

5.4 spfa 算法

5.5 【模板】单源最短路问题（弱化版）

> **题目来源：** 洛谷
>
> **题目链接：** P3371 【模板】单源最短路径（弱化版）
>
> **难度系数：** ★★

【题目描述】

> 给出一个有向图，请输出从某一点出发到所有点的最短路径长度。

【输入描述】

> 第一行包含三个整数 n, m, s，分别表示点的个数、有向边的个数、出发点的编号。
>
> 接下来 m 行每行包含三个整数 u, v, w，表示一条 $u \to v$ 的，长度为 w 的边。

【输出描述】

> 输出一行 n 个整数，第 i 个表示 s 到第 i 个点的最短路径，若不能到达则输出 $2^{31}-1$。

【示例一】

> 输入：
>
> ```
> 4 6 1
> 1 2 2
> 2 3 2
> 2 4 1
> 1 3 5
> 3 4 3
> 1 4 4
> ```
>
> 输出：
>
> ```
> 0 2 4 3
> ```

【解法】

常规版 dijkstra 算法。

【参考代码】

```
#include <iostream>
#include <vector>
#include <cmath>

using namespace std;

typedef pair<int, int> PII;
typedef long long LL;

const int N = 1e4 + 10, INF = 2147483647;

int n, m, s;
vector<PII> edges[N];

LL dist[N];
bool st[N];

void dijkstra()
{
```

```cpp
    for(int i = 0; i <= n; i++) dist[i] = INF;
    dist[s] = 0;

    for(int i = 1; i < n; i++)
    {
        int u = 0;
        for(int j = 1; j <= n; j++)
        {
            if(!st[j] && dist[j] < dist[u]) u = j;
        }

        st[u] = true;

        for(auto& t : edges[u])
        {
            int v = t.first, w = t.second;
            if(dist[u] + w < dist[v])
            {
                dist[v] = dist[u] + w;
            }
        }
    }

    for(int i = 1; i <= n; i++)
    {
        cout << dist[i] << " ";
    }
}

int main()
{
    cin >> n >> m >> s;
    for(int i = 1; i <= m; i++)
    {
        int u, v, w; cin >> u >> v >> w;
        edges[u].push_back({v, w});
    }

    dijkstra();

    return 0;
}
```

5.6 【模板】单源最短路问题（标准版）

> **题目来源：** 洛谷
>
> **题目链接：** P4779 【模板】单源最短路径（标准版）
>
> **难度系数：** ★★

【题目描述】

> 给定一个 n 个点，m 条有向边的带非负权图，请你计算从 s 出发，到每个点的距离。
>
> 数据保证你能从 s 出发到任意点。

【输入描述】

> 第一行为三个正整数 n, m, s。 第二行起 m 行，每行三个非负整数 u_i, v_i, w_i，表示从 u_i 到 v_i 有一条权值为 w_i 的有向边。

【输出描述】

> 输出一行 n 个空格分隔的非负整数，表示 s 到每个点的距离。

【示例一】

> 输入：
>
> ```
> 4 6 1
> 1 2 2
> 2 3 2
> 2 4 1
> 1 3 5
> 3 4 3
> 1 4 4
> ```
>
> 输出：
>
> ```
> 0 2 4 3
> ```

【解法】

【参考代码】

```
1
```

5.7 【模板】负环

题目来源： 洛谷

题目链接： P3385 【模板】负环

难度系数： ★★

【题目描述】

给定一个 n 个点的有向图，请求出图中是否存在**从顶点 1 出发能到达**的负环。

负环的定义是：一条边权之和为负数的回路。

【输入描述】

本题单测试点有多组测试数据。

输入的第一行是一个整数 T，表示测试数据的组数。对于每组数据的格式如下：

第一行有两个整数，分别表示图的点数 n 和接下来给出边信息的条数 m。

接下来 m 行，每行三个整数 u, v, w。

- 若 $w \geq 0$，则表示存在一条从 u 至 v 边权为 w 的边，还存在一条从 v 至 u 边权为 w 的边。
- 若 $w < 0$，则只表示存在一条从 u 至 v 边权为 w 的边。

【输出描述】

对于每组数据，输出一行一个字符串，若所求负环存在，则输出 `YES`，否则输出 `NO`。

【示例一】

输入：

```
2
3 4
1 2 2
1 3 4
2 3 1
3 1 -3
3 3
1 2 3
2 3 4
3 1 -8
```

输出：

```
NO
```

YES

【解法】

【参考代码】

```
1
```

5.8 邮递员送信

题目来源： 洛谷

题目链接： P1629 邮递员送信

难度系数： ★★

【题目描述】

有一个邮递员要送东西，邮局在节点 11。他总共要送 n−1$n-1$ 样东西，其目的地分别是节点 22 到节点 nn。由于这个城市的交通比较繁忙，因此所有的道路都是单行的，共有 mm 条道路。这个邮递员每次只能带一样东西，并且**运送每件物品过后必须返回邮局**。求送完这 n−1$n-1$ 样东西并且**最终回到邮局**最少需要的时间。

【输入描述】

第一行包括两个整数，n 和 m，表示城市的节点数量和道路数量。

第二行到第 $(m+1)$ 行，每行三个整数，u,v,w，表示从 u 到 v 有一条通过时间为 w 的道路。

【输出描述】

输出仅一行，包含一个整数，为最少需要的时间。

【示例一】

输入：

```
5 10
2 3 5
1 5 5
3 5 6
1 2 8
1 3 8
```

```
5 3 4
4 1 8
4 5 3
3 5 6
5 4 2
```

输出：

```
83
```

【解法】

【参考代码】

```
1
```

5.9 采购特价商品

题目来源： 洛谷

题目链接： P1744 采购特价商品

难度系数： ★★

【题目描述】

中山路店山店海，成了购物狂爱与愁大神的“不归之路”。中山路上有 $n(n \leq 100)$ 家店，每家店的坐标均在 -10000 至 10000 之间。其中的 m 家店之间有通路。若有通路，则表示可以从一家店走到另一家店，通路的距离为两点间的直线距离。现在爱与愁大神要找出从一家店到另一家店之间的最短距离。你能帮爱与愁大神算出吗?

【输入描述】

共 $n+m+3$ 行：

第一行：整数 n。

接下来 n 行：每行两个整数 x 和 y，描述了一家店的坐标。

接下来一行：整数 m。

接下来 m 行：每行描述一条通路，由两个整数 i 和 j 组成，表示第 i 家店和第 j 家店之间有通路。

> 接下来一行：两个整数 s 和 t ，分别表示原点和目标店。

【输出描述】

> 仅一行：一个实数（保留两位小数），表示从 s 到 t 的最短路径长度。

【示例一】

> 输入：
>
> 5
>
> 0 0
>
> 2 0
>
> 2 2
>
> 0 2
>
> 3 1
>
> 5
>
> 1 2
>
> 1 3
>
> 1 4
>
> 2 5
>
> 3 5
>
> 1 5
>
> 输出：
>
> 3.41

【解法】

【参考代码】

```
1
```

5.10 拉近距离

> **题目来源： 洛谷**

> **题目链接：** P2136 拉近距离
>
> **难度系数：** ★★★

【题目描述】

> 在小明和小红的生活中，有 N 个关键的节点。有 M 个事件，记为一个三元组 (S_i, T_i, W_i)，表示从节点 S_i 有一个事件可以转移到 T_i，事件的效果就是使他们之间的距离减少 W_i。
>
> 这些节点构成了一个网络，其中节点 1 和 N 是特殊的，节点 1 代表小明，节点 N 代表小红，其他代表进展的阶段。所有事件可以自由选择是否进行，但每次只能进行当前节点邻接的。请你帮他们写一个程序，计算出他们之间可能的最短距离。

【输入描述】

> 第一行，两个正整数 N, M。
>
> 之后 M 行，每行 3 个空格隔开的整数 S_i, T_i, W_i。

【输出描述】

> 一行，一个整数表示他们之间可能的最短距离。如果这个距离可以无限缩小，输出 `Forever love`。

【示例一】

> 输入：
>
> ```
> 3 3
> 1 2 3
> 2 3 -1
> 3 1 -10
> ```
>
> 输出：
>
> ```
> -2
> ```

【解法】

【参考代码】

```
1
```

5.11 最短路计数

> **题目来源：** 洛谷

> **题目链接：** P1144 最短路计数
>
> **难度系数：** ★★★

【题目描述】

> 给出一个 N 个顶点 M 条边的无向无权图，顶点编号为 $1 \sim N$。问从顶点 1 开始，到其他每个点的最短路有几条。

【输入描述】

> 第一行包含 2 个正整数 N, M，为图的顶点数与边数。
>
> 接下来 M 行，每行 2 个正整数 x, y，表示有一条连接顶点 x 和顶点 y 的边，请注意可能有自环与重边。

【输出描述】

> 共 N 行，每行一个非负整数，第 i 行输出从顶点 1 到顶点 i 有多少条不同的最短路，由于答案有可能会很大，你只需要输出 $ans \mod 100003$ 后的结果即可。如果无法到达顶点 i 则输出 0。

【示例一】

> 输入：
>
> ```
> 5 7
> 1 2
> 1 3
> 2 4
> 3 4
> 2 3
> 4 5
> 4 5
> ```
>
> 输出：
>
> ```
> 1
> 1
> 1
> 2
> 4
> ```

【解法】

【参考代码】

```
1
```

6. 多源最短路

6.1 【模板】floyd

题目来源： 洛谷

题目链接： B3647 【模板】Floyd

难度系数： ★

【题目描述】

给出一张由 n 个点 m 条边组成的无向图。

求出所有点对 (i,j) 之间的最短路径。

【输入描述】

第一行为两个整数 n,m，分别代表点的个数和边的条数。

接下来 m 行，每行三个整数 u,v,w，代表 u,v 之间存在一条边权为 w 的边。

【输出描述】

输出 n 行每行 n 个整数。

第 i 行的第 j 个整数代表从 i 到 j 的最短路径。

【示例一】

输入：

```
4 4
1 2 1
2 3 1
3 4 1
4 1 1
```

输出：

```
0 1 2 1
```

```
1012
2101
1210
```

【解法】

【参考代码】

```
1
```

6.2 Clear And Present Danger

题目来源： 洛谷

题目链接： P2910 [USACO08OPEN] Clear And Present Danger S

难度系数： ★★

【题目描述】

农夫约翰正驾驶一条小艇在牛勒比海上航行.

海上有 $N(1 \leq N \leq 100)$ 个岛屿，用 1 到 N 编号．约翰从 1 号小岛出发，最后到达 N 号小岛.

一张藏宝图上说，如果他的路程上经过的小岛依次出现了 $A_1, A_2, \ldots, A_M (2 \leq M \leq 10000)$ 这样的序列（不一定相邻），那他最终就能找到古老的宝藏． 但是，由于牛勒比海有海盗出没．约翰知道任意两个岛屿之间的航线上海盗出没的概率，他用一个危险指数 $D_{i,j} (0 \leq D_{i,j} \leq 100000)$ 来描述．他希望他的寻宝活动经过的航线危险指数之和最小．那么，在找到宝藏的前提下，这个最小的危险指数是多少呢?

【输入描述】

第一行：两个用空格隔开的正整数 N 和 M 。

第二到第 $M+1$ 行：第 $i+1$ 行用一个整数 A_i 表示 FJ 必须经过的第 i 个岛屿

第 $M+2$ 到第 $N+M+1$ 行：第 $i+M+1$ 行包含 N 个用空格隔开的非负整数分别表示 i 号小岛到第 $1 \ldots N$ 号小岛的航线各自的危险指数。保证第 i 个数是 0 。

【输出描述】

FJ 在找到宝藏的前提下经过的航线的危险指数之和的最小值。

【示例一】

输入：

```
3 4
1
2
1
3
0 5 1
5 0 2
1 2 0
```

输出：

```
7
```

【解法】

【参考代码】

```
1
```

6.3 灾后重建

题目来源： 洛谷

题目链接： P1119 灾后重建

难度系数： ★★★

【题目描述】

给出 B 地区的村庄数 N，村庄编号从 0 到 $N-1$，和所有 M 条公路的长度，公路是双向的。并给出第 i 个村庄重建完成的时间 t_i，你可以认为是同时开始重建并在第 t_i 天重建完成，并且在当天即可通车。若 t_i 为 0 则说明地震未对此地区造成损坏，一开始就可以通车。之后有 Q 个询问 (x,y,t)，对于每个询问你要回答在第 t 天，从村庄 x 到村庄 y 的最短路径长度为多少。如果无法找到从 x 村庄到 y 村庄的路径，经过若干个已重建完成的村庄，或者村庄 x 或村庄 y 在第 t 天仍未重建完成，则需要输出 -1。

【输入描述】

第一行包含两个正整数 N, M，表示了村庄的数目与公路的数量。

第二行包含 N 个非负整数 $t_0, t_1, \cdots, t_{N-1}$，表示了每个村庄重建完成的时间，数据保证了 $t_0 \leq t_1 \leq \cdots \leq t_{N-1}$。

接下来 M 行，每行 3 个非负整数 i, j, w，w 不超过 10000，表示了有一条连接村庄 i 与村庄 j 的道路，长度为 w，保证 $i \neq j$，且对于任意一对村庄只会存在一条道路。

接下来一行也就是 $M+3$ 行包含一个正整数 Q，表示 Q 个询问。

接下来 Q 行，每行 3 个非负整数 x, y, t，询问在第 t 天，从村庄 x 到村庄 y 的最短路径长度为多少，数据保证了 t 是不下降的。

【输出描述】

共 Q 行，对每一个询问 (x, y, t) 输出对应的答案，即在第 t 天，从村庄 x 到村庄 y 的最短路径长度为多少。如果在第 t 天无法找到从 x 村庄到 y 村庄的路径，经过若干个已重建完成的村庄，或者村庄 x 或村庄 y 在第 t 天仍未修复完成，则输出 -1。

【示例一】

输入：

```
4 5
1 2 3 4
0 2 1
2 3 1
3 1 2
2 1 4
0 3 5
4
2 0 2
0 1 2
0 1 3
0 1 4
```

输出：

```
-1
-1
5
4
```

【解法】

【参考代码】

```
1
```

6.4 无向图的最小环问题

> **题目来源：** 洛谷
>
> **题目链接：** P6175 无向图的最小环问题
>
> **难度系数：** ★★

【题目描述】

> 给定一张无向图，求图中一个至少包含 3 个点的环，环上的节点不重复，并且环上的边的长度之和最小。该问题称为无向图的最小环问题。在本题中，你需要输出最小的环的边权和。若无解，输出 `No solution.`。

【输入描述】

> 第一行两个正整数 n, m 表示点数和边数。
>
> 接下来 m 行，每行三个正整数 u, v, d，表示节点 u, v 之间有一条长度为 d 的边。

【输出描述】

> 输出边权和最小的环的边权和。若无解，输出 `No solution.`。

【示例一】

> 输入：
>
> 5 7
>
> 1 4 1
>
> 1 3 300
>
> 3 1 10
>
> 1 2 16
>
> 2 3 100
>
> 2 5 15
>
> 5 3 20

> 输出：
>
> 61

【解法】

【参考代码】

```
1
```